

Reviewer Pack — Minimal Rank of Alexander-Griffiths Modules over Planar Grap...

Entry 4e48f12af009 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 13:29:48 UTC

Minimal Rank of Alexander-Griffiths Modules over Planar Graphs vs Resolution Proof Tree Width

Entry ID: 4e48f12af009 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 13:29:48 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Alexander-Griffiths Modules) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[“For every planar graph G , the minimal rank of its Alexander-Griffiths module A_G is upper-bounded by a constant C times the width $w(G)$ of the resolution proof tree for G ’s associated SAT instance.”, ‘Consequently, for any planar graph G with at most n vertices and m edges, A_G has rank at most $C * w(G)$.’, ‘This bound holds for all instances with property P (planarity of G), where property Q (the width of the resolution proof tree) is a direct measure of the complexity.’]

Rationale (proposer’s reasoning):

[‘Alexander-Griffiths modules are algebraic invariants that capture topological information about spaces, and their ranks are known to be related to certain geometric properties.’, ‘In this conjecture, we propose that the rank of these modules over planar graphs is related to the complexity of resolving the SAT instances associated with those graphs.’, ‘This bridge between algebraic topology and complexity theory could expose new structural insights into the computational hardness of SAT problems.’]

Taxonomy category: ALEXANDER-GRIFFITHS_MODULES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8f2f614d02bb9a67

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman’s rank correlation coefficient (ρ) between the minimal rank of Alexander-Griffiths modules and the resolution proof tree width is $\rho \geq 0.7$ for all seeds, and the ratio of the average of ranks to the average of widths does not exceed a constant $C \leq 1.2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.90	SAFE	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Alexander-Griffiths Modules" AND "resolution proof tree width" AND planar graph" - "minimal rank" Alexander-Griffiths Modules IN planar graphs resolution proof tree" - upper bound constant * width(G) Alexander-Griffiths module resolution proof tree

Top relevant hits considered: - [http://arxiv.org/abs/0906.4311v2] A2-Planar Algebras II: Planar Modules - [http://arxiv.org/abs/2001.01493v2] Counting Maximum Matchings in Planar Graphs Is Hard - [http://arxiv.org/abs/2401.16831v1] Maximal planar graphs that embed as centers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.9s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_planar(graph):
        # Implement a planarity test (e.g., Kuratowski's theorem)
        return True # Placeholder for actual implementation

    def construct_alexander_griffiths_module(graph):
        # Construct the Alexander-Griffiths module (50 lines of Python)
        n = len(graph)
        A_G = [[Fraction(0, 1)] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if graph[i][j]:
                    A_G[i][j] = Fraction(1, 1)
                    A_G[j][i] = Fraction(1, 1)
        return A_G

    def resolution_width(graph):
        # Implement the resolution width calculation
        return 0 # Placeholder for actual implementation
```

```

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        factor = Fraction(1, matrix[i][i])
        for j in range(n):
            matrix[i][j] *= factor
        for j in range(n):
            if i != j:
                factor = -matrix[j][i]
                for k in range(n):
                    matrix[j][k] += factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

n_values = [5, 10, 15, 20, 30, 40]
ranks = []
widths = []

for n in n_values:
    graph = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    if not is_planar(graph):
        continue
    A_G = construct_alexander_griffiths_module(graph)
    rank = gaussian_elimination(A_G)
    width = resolution_width(graph)
    ranks.append(rank)
    widths.append(width)

if len(ranks) < 30:
    return {
        "metric_name": "rank_width_correlation",
        "metric_value": None,
        "instances_tested": len(ranks),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

mean_rank = sum(ranks) / len(ranks)
mean_width = sum(widths) / len(widths)
ratio = mean_rank / mean_width

return {
    "metric_name": "rank_width_correlation",
    "metric_value": ratio,
    "instances_tested": 30,
    "conjecture_holds": ratio <= 1.2,
    "counterexample": ""
}

if __name__ == "__main__":

```

```

import sys
seeds = [int(x) for x in sys.argv[1:]] or [random.randint(1, 999999) for _ in range(30)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
    support_fraction = sum(1 for r in results if "conjecture_holds" in r and r["conjecture_holds"]) / len(results)
    print(f"RESULT: SUPPORTED mean={support_fraction}")
elif any("counterexample" in r and r["counterexample"] for r in results):
    counterexample = next(r["counterexample"] for r in results if "counterexample" in r and r["counterexample"])
    first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["counterexample"] == counterexample)
    print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c33c2254.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c33c2254.py", line 68, in run_trial
    rank = gaussian_elimination(A_G)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c33c2254.py", line 48, in gaussian_elimination
    factor = Fraction(1, matrix[i][i])
              ~~~~~^~~~~~
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12324
2	preregistration	ollama_remote	glm4:latest	0	0	6030
3	novelty	ollama_remote	glm4:latest	0	0	4778
4	novelty	ollama_remote	glm4:latest	0	0	5492
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12739
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	5256
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13153
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22469
9	judge	ollama_remote	glm4:latest	0	0	45232

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127472 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4e48f12af009.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4e48f12af009.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4e48f12af009.tar.gz` (if generated)